pypedigree

Generate publication-quality pedigree diagrams from standard genetics pedigree files

v0.4.0

Installation

From PyPI

```
pip install pypedigree
```

With pandas support

```
pip install pypedigree[pandas]
```

From source (development)

```
git clone https://github.com/darkxander/Pypedigree.git
cd pypedigree
uv sync --extra dev
```

Requirements

- Python 3.10+
- matplotlib ≥ 3.7
- numpy ≥ 1.24
- click ≥ 8.0 (for CLI)
- pandas (optional, for DataFrame input)

Quick Start

Command Line

```
# Render a pedigree file to PNG
pypedigree family.ped -o output.png

# Display interactively
pypedigree family.ped --show

# Render specific family at high resolution
pypedigree family.ped --family FAM1 --dpi 300 -o family1.svg
```

Python Script

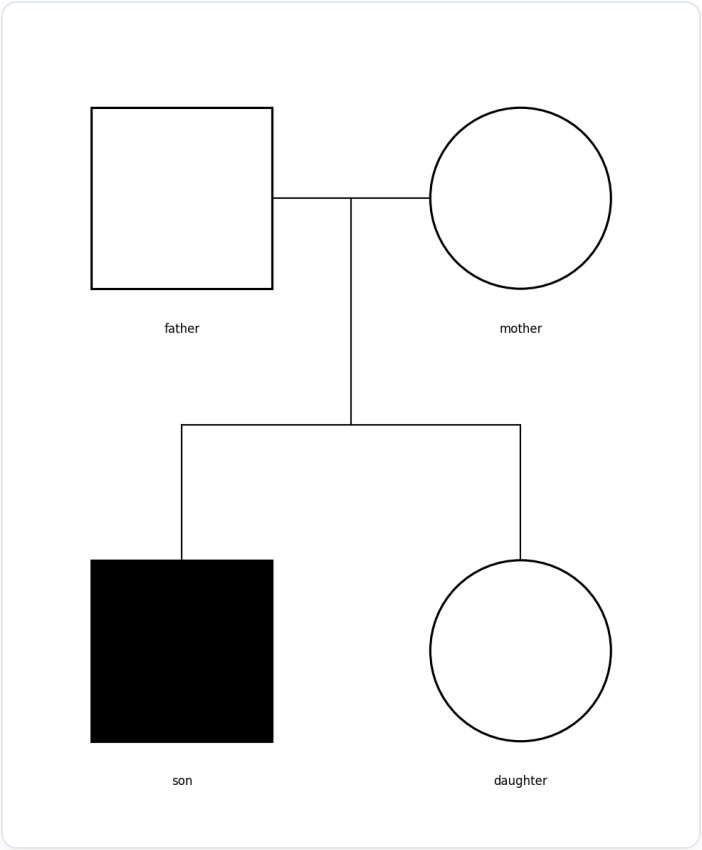
```
from pypedigree import parse_pedigree, render_pedigree

ped = parse_pedigree("family.ped")
fig = render_pedigree(ped, output="output.png")
```

Programmatic Builder

```
from pypedigree import PedigreeBuilder

builder = PedigreeBuilder("FAM1")
builder.add_individual("father", sex="male")
builder.add_individual("mother", sex="female")
builder.add_individual("child", sex="male",
                       father="father", mother="mother",
                       phenotype="affected")
fig = builder.render(output="trio.png")
```



Output of the PedigreeBuilder example above

Pedigree File Format

pypedigree reads tab-, comma-, or space-separated pedigree files. The separator is auto-detected.

Smart Header-Based Column Mapping

When a header row is present, columns are matched **by name, not by position**. This means you can arrange columns in any order, and any column that doesn't match a known structural name is automatically treated as a phenotype column.

The parser recognises multiple aliases for each structural column:

ROLE	ACCEPTED ALIASES
Family ID	`FID`, `family_id`, `Family`
Individual ID	`ID`, `IID`, `sample_id`, `Individual`
Father	`Father`, `Pat`, `Dad`, `paternal_id`
Mother	`Mother`, `Mat`, `Mom`, `Mum`, `maternal_id`
Sex	`Sex`, `Gender`
Phenotype	`Phenotype`, `Pheno`, `Affected`

Extra columns become phenotypes

Any header name that doesn't match one of the structural aliases above is automatically parsed as an extra phenotype column. This makes it easy to add phenotype data without worrying about column positions.

Standard 6-Column Format

COLUMN	NAME	DESCRIPTION	VALUES
1	family_id	Family identifier	Any string

COLUMN	NAME	DESCRIPTION	VALUES
2	sample_id	Individual identifier (unique within family)	Any string
3	paternal_id	Father's sample_id	ID or `0` if unknown
4	maternal_id	Mother's sample_id	ID or `0` if unknown
5	sex	Biological sex	`1`=male, `2`=female, `0`=unknown
6	phenotype	Affected status	`1`=unaffected, `2`=affected, `0`=unknown

Example: simple_trio.ped

```
FAM1  father  0       0       1  1
FAM1  mother  0       0       2  1
FAM1  child   father  mother  1  2
```

Extended Format (Columns 7–10)

COLUMN	NAME	DESCRIPTION	VALUES
7	date_of_birth	Birth date	`YYYYMMDD`, `YYYYMM`, or `YYYY`
8	twins	Twin relationships	`TYPE:sibling_id` (comma-separated)
9	deceased	Vital status	`1`=deceased, `0`=alive
10	proband	Index case	`1`=proband, `0`=not
11	carrier	Carrier status	`0`=not carrier, `1`=carrier (heterozygous), `2`=obligate carrier. Per-gene status: `cr_` columns, see Carrier Notation

Standard notation — pregnancies and pregnancy losses, adoption, childlessness, gamete donors and surrogates, gender identity, the consultand — has its own named columns, see Standard

Notation.

Carrier column

The `carrier` column requires a header row to be present. When using inline phenotypes (columns 12+), the carrier column must appear before the phenotype columns.

Twin types

`MZ` = monozygotic (identical), `DZ` = dizygotic (fraternal), `TW` = unknown zygosity.
Example: `MZ:sibling_id`

Inline Phenotypes (Columns 11+)

When a **header row** is present, columns 11 and beyond are parsed as additional phenotype values. Column names from the header define phenotype names.

Example with inline phenotypes

family_id	sample_id	father	mother	sex	pheno	dob	twins	deceased	proband
FAM1	father	0	0	1	1	0	0	0	0
FAM1	mother	0	0	2	1	0	0	0	0
FAM1	son	father	mother	1	2	0	0	0	1
FAM1	daughter	father	mother	2	1	0	0	0	0

Phenotype value interpretation

- **Standard pedigree encoding (1/2)**
  : values `1` =unaffected, `2` =affected, `0` or `-9` =unknown are properly handled in extra phenotype columns, following the same convention as the main phenotype column
- **Binary**
  (0 or 1): filled band (1) or empty band (0)
- **Continuous**
  (values > 1): color gradient from white (min) to full color (max)
- **PGS column**
  : rendered as a vertical Gaussian distribution curve
- **Missing**
  (`.` , `NA` , `NaN` , `-9` , empty): white/blank band

- `a_` **prefix columns**

  are annotation columns (text below IDs, not phenotypes)
- `gi_` **prefix**

  for gradient inverted (color→white, for metrics where low = bad)
- `cr_` **prefix**

  for carrier status of one gene or condition (0/1/2), drawn as a patterned section, see Carrier Notation
- **Comma decimal separator**

  : `0,712` is accepted as `0.712`

Binary phenotype rendering

Binary phenotypes (including standard 1/2 encoding) are rendered as:
color

=affected,
white

=unaffected,
grey

=unknown. No text labels are shown by default. To display YES/NO text inside the bands, use the `show_phenotype_labels` style option or the `--show-phenotype-labels` CLI flag.

Separate Phenotype File

Extra phenotypes can also be supplied via a separate TSV file with the `--pheno` CLI flag:

```
sample_id  epilepsy  IQ   SRS
father     0         110  12
mother     1         95   25
son        1         85   40
daughter   0         120  8
```

CLI Reference

```
pypedigree PEDIGREE_FILE [OPTIONS]
```

OPTION	DESCRIPTION	DEFAULT
`-o, --output PATH`	Output file path (format from extension: .png, .svg, .pdf)	None (display)
`--format FORMAT`	Force output format: `png`, `svg`, `pdf`	Auto-detect
`--family ID`	Render only this family	All families
`--dpi INT`	Output resolution in DPI	150
`--show`	Display interactively (matplotlib window)	Off
`--style PATH`	JSON file with PedigreeStyle overrides	None
`--separator SEP`	Force separator: `tab`, `comma`, `space`	Auto-detect
`--pheno PATH`	Separate .pheno TSV file for extra phenotypes	None
`--annotations PATH`	Separate TSV file with annotation text (displayed below individual IDs)	None
`--outline-color COLOR`	Shape outline color (e.g. `grey`, `#888`)	black
`--carrier-notation [nsgc2022|legacy]`	Carrier notation: patterned sections (NSGC 2022) or half-fill and center dot	nsgc2022
`--no-labels`	Hide sample ID labels under shapes	Off
`--no-legend`	Hide phenotype legend	Off
`--show-phenotype-labels`	Show YES/NO text inside binary phenotype bands (hidden by default)	Off

OPTION	DESCRIPTION	DEFAULT
`--plink`	Treat input as PLINK .ped file (ignore genotype columns after column 6)	Off
`--interactive`	Generate interactive SVG with tooltips, click highlighting, and zoom/pan (same bands, legend and PGS curves as the PNG; not with `--circular`)	Off
`--circular`	Use circular/radial layout with founders at centre, descendants on concentric rings	Off
`--version`	Show version and exit	—

Examples

```
# Basic rendering
pypedigree family.ped -o pedigree.png

# SVG with custom DPI and no labels
pypedigree family.ped -o pedigree.svg --dpi 300 --no-labels

# With external phenotype file and custom style
pypedigree family.ped --pheno phenotypes.tsv --style custom.json -o result.pdf

# Grey outlines for a cleaner look
pypedigree family.ped --outline-color grey -o pedigree.png

# Interactive SVG with tooltips and click highlighting
pypedigree family.ped --interactive -o pedigree.svg

# Show YES/NO labels in binary phenotype bands
pypedigree family.ped --show-phenotype-labels -o pedigree.png

# Read a PLINK .fam file (auto-detected from extension)
pypedigree family.fam -o pedigree.png

# Read a PLINK .ped file (use --plink to ignore genotype columns)
pypedigree family.ped --plink -o pedigree.png
```

Python API

Parsing

`parse_pedigree(file_path)`

Parse a pedigree file (.ped) into a `Pedigree` object.

```
from pypedigree import parse_pedigree

ped = parse_pedigree("family.ped")
for fam_id, family in ped.families.items():
    print(f"{fam_id}: {len(family.individuals)} individuals")
```

`parse_pedigree_from_string(text)`

Parse pedigree data from a string (useful for testing or embedded data).

```
from pypedigree import parse_pedigree_from_string

data = """FAM1\tfather\t0\t0\t1\t1
FAM1\tmother\t0\t0\t2\t1
FAM1\tchild\tfather\tmother\t1\t2"""
ped = parse_pedigree_from_string(data)
```

`parse_pedigree_from_dataframe(df)`

Parse from a pandas DataFrame. Requires `pip install pypedigree[pandas]`.

```
import pandas as pd
from pypedigree import parse_pedigree_from_dataframe

df = pd.read_csv("family.ped", sep="\t", header=None)
ped = parse_pedigree_from_dataframe(df)
```

`parse_phenotype_file(file_path)`

Parse a separate phenotype TSV file.

```
from pypedigree import parse_phenotype_file, apply_phenotypes_to_pedigree
```

```
pheno_data, pheno_names = parse_phenotype_file("phenotypes.tsv")
apply_phenotypes_to_pedigree(ped, pheno_data, pheno_names)
```

`parse_annotation_file(file_path)`

Parse an external annotation TSV file. First row is a header (`sample_id`, `ann1`, `ann2`, ...), subsequent rows contain text values.

```
from pypedigree import parse_annotation_file, apply_annotations_to_pedigree

ann_data, ann_names = parse_annotation_file("annotations.tsv")
apply_annotations_to_pedigree(ped, ann_data, ann_names)
```

Rendering

`render_pedigree(pedigree, ...)`

Render all families in a pedigree. Returns a matplotlib `Figure`.

`pedigree: Pedigree`
: The pedigree to render.

`style: PedigreeStyle | None = None`
: Visual style (defaults if None).

`output: str | Path | None = None`
: Save to file (PNG/SVG/PDF). None for in-memory only.

`ax: matplotlib.axes.Axes | None = None`
: Render onto existing axes.

`show_labels: bool = True`
: Show sample ID labels under shapes.

`show_legend: bool = True`
: Show phenotype legend (when phenotypes + colors exist).

`circular: bool = False`
: Use circular/radial layout (founders at centre, descendants on concentric rings).

```
from pypedigree import parse_pedigree, render_pedigree, PedigreeStyle

ped = parse_pedigree("family.ped")
style = PedigreeStyle(dpi=300)
fig = render_pedigree(ped, style=style, output="high_res.png")
```

`render_family(family, ...)`

Render a single `Family` object. Returns `(Figure, FamilyLayout)`.

```
from pypedigree import parse_pedigree, render_family

ped = parse_pedigree("family.ped")
family = list(ped.families.values())[0]
fig, layout = render_family(family)
fig.savefig("family.png", dpi=150, bbox_inches="tight")
```

`render_interactive_svg(pedigree, ...)`

Render all families as an interactive SVG with tooltips, click highlighting, and zoom/pan. Returns the SVG markup as a string. Draws the same figure as `render_pedigree()`. No additional dependencies required.

`pedigree: Pedigree`
: The pedigree to render.

`style: PedigreeStyle | None = None`
: Visual style (defaults if None).

`output: str | Path | None = None`
: Save SVG to file. None for in-memory only.

`show_labels: bool = True`
: Draw sample ID labels under shapes.

`show_legend: bool = True`
: Draw the phenotype and carrier legend.

```
from pypedigree import parse_pedigree, render_interactive_svg

ped = parse_pedigree("family.ped")
svg_str = render_interactive_svg(ped, output="interactive.svg")
```

Layout (advanced)

`compute_layout(family, style)`

Compute layout coordinates without rendering. Returns a `FamilyLayout` with node positions and edge geometry.

```python
from pypedigree import compute_layout, PedigreeStyle

layout = compute_layout(family, PedigreeStyle())
for node in layout.nodes:
    print(f"{node.individual_id}: x={node.x:.1f}, y={node.y:.1f}")
```

`compute_layout_circular(family, style)`

Compute circular/radial layout coordinates. Founders sit at the centre and each later generation on its own concentric ring, in the left-to-right order of the hierarchical layout wrapped around the circle. The circular layout therefore inherits its guarantees: siblings stay together, sibship arcs of different couples never overlap, partners sit side by side, and tangled loops are drawn with duplicated individuals.

```python
from pypedigree import compute_layout_circular, PedigreeStyle

layout = compute_layout_circular(family, PedigreeStyle())
for node in layout.nodes:
    print(f"{node.individual_id}: x={node.x:.1f}, y={node.y:.1f}")
```

`base_radius` is expanded automatically when a ring would be too crowded for the shapes to fit, so nodes never overlap. Shapes stay upright: a square turned to face outward would read as the unknown-sex diamond, and phenotype bands stay horizontal as in the hierarchical layout. ID labels are set radially outside each shape, away from the lines arriving from the centre, on a white backing. Twins hang from one point of the sibship arc, as a V with a bar for MZ twins, and duplicates are joined to their original by a dotted arc.

PedigreeBuilder API

The `PedigreeBuilder` provides a fluent interface for constructing pedigrees programmatically, without writing .ped files. All methods return `self` for method chaining.

`PedigreeBuilder(family_id="FAM1")`

Create a new builder with a default family ID.

`add_individual(sample_id, ...)`

Add an individual to the pedigree.

`sample_id: str`
: Unique identifier for this individual (required).

`sex: str | Sex = "unknown"`
: Biological sex: `"male"`, `"female"`, `"unknown"` (or `"m"`, `"f"`, `"u"`, `"1"`, `"2"`, `"0"`).

`phenotype: str | Phenotype = "unknown"`
: Affected status: `"affected"`, `"unaffected"`, `"unknown"`.

`father: str | None = None`
: Father's sample_id. Phantom parent created if not added explicitly.

`mother: str | None = None`
: Mother's sample_id. Phantom parent created if not added explicitly.

`family_id: str | None = None`
: Override the builder's default family ID.

`deceased: bool = False`
: Whether this individual is deceased (diagonal line through shape).

`proband: bool = False`
: Whether this is the proband/index case (arrow marker).

`date_of_birth: str | None = None`
: Date of birth string.

`phenotypes: dict[str, float | int] | None = None`
: Extra phenotype values as {name: value}.

`carrier: bool = False`
: Whether this individual is a carrier (heterozygous). Rendered as a patterned section (NSGC 2022), or a half-filled shape with the legacy notation.

`obligate_carrier: bool = False`

Whether this is an obligate carrier. Rendered as a sparser pattern (NSGC 2022), or a center dot with the legacy notation.

`annotations: list[str] | None = None`

Text lines to display below the individual's ID label (e.g. variant names).

`pregnancy: str | Pregnancy | None = None`

`"P"`, `"SAB"`, `"TOP"`, `"ECT"` or `"SB"`, see Standard Notation.

`adopted: str | Adoption | None = None`

`"in"` (father/mother are the adoptive parents) or `"out"`.

`childless: str | Childlessness | None = None`

`"choice"` or `"infertility"`.

`art: str | ReproductiveRole | None = None`

`"D"` gamete donor or `"S"` surrogate.

`gender: str | Gender | None = None`

Gender identity, `"man"`, `"woman"` or `"nonbinary"`: the symbol follows it, and `sex` is the sex assigned at birth.

`consultand: bool = False`

Arrow without “P”.

`add_union(partner1, partner2, ...)`

Explicitly add a union between two individuals. Unions are auto-inferred from parent references, so this is only needed for **childless couples** or to set consanguinity/broken flags.

`partner1: str`

First partner's sample_id.

`partner2: str`

Second partner's sample_id.

`consanguineous: bool = False`

Double-line relationship (shared ancestors).

`broken: bool = False`

Broken-line relationship (dissolved).

`set_style(style)`

Set a custom `PedigreeStyle` for rendering.

`build() → Pedigree`

Build a `Pedigree` object from added individuals. Auto-infers unions and creates phantom parents.

`render(output=None, ...) → Figure`

Build and render in one step. Auto-assigns phenotype colors if not set.

`render_interactive_svg(output=None) → str`

Build and render as an interactive SVG. Returns the SVG markup as a string. If `output` is provided, the SVG is also written to that file path. See the Interactive SVG section for details.

Complete Example

```
from pypedigree import PedigreeBuilder

builder = PedigreeBuilder("FAM1")

# Generation 1: founders
builder.add_individual("gf", sex="male")
builder.add_individual("gm", sex="female")

# Generation 2
builder.add_individual("father", sex="male", father="gf", mother="gm")
builder.add_individual("mother", sex="female")

# Generation 3 with phenotypes
builder.add_individual("child1", sex="male",
    father="father", mother="mother",
    phenotype="affected", proband=True,
    phenotypes={"IQ": 85, "epilepsy": 1})
builder.add_individual("child2", sex="female",
    father="father", mother="mother",
    phenotypes={"IQ": 120, "epilepsy": 0})

fig = builder.render(output="family.png")
```

Jupyter Notebook Integration

The builder has built-in Jupyter support via `_repr_png_`. Simply evaluate a builder cell to see the pedigree inline:

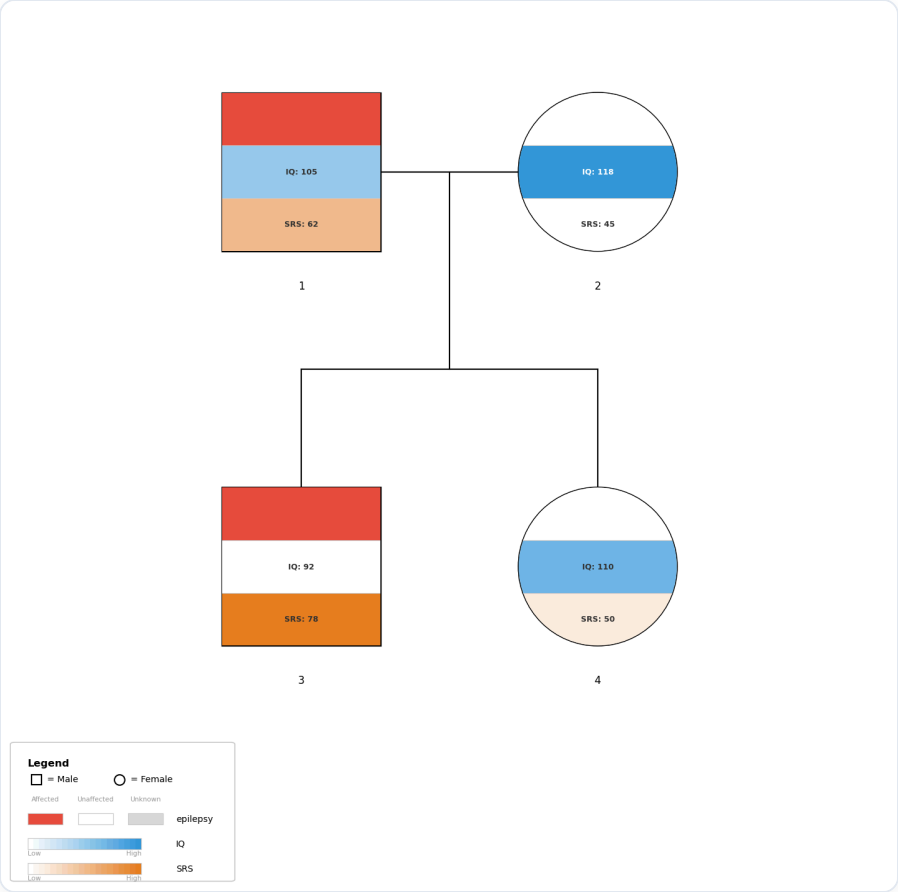
```
builder  # Displays the pedigree diagram inline
```

Multi-Phenotype Support

pypedigree can display multiple phenotypes per individual using colored bands or gradient fills on each shape.

Phenotype Types

TYPE	VALUES	RENDERING
Binary	0 or 1	Empty (0) or filled (1) color band
Continuous	Numeric > 1	Color gradient: white (min) → full color (max)
PGS	Any numeric	Vertical Gaussian distribution curve (to the left of shape)
Missing	`.`, `NA`, `NaN`, `-9`, empty	White/blank band



Multi-phenotype pedigree with epilepsy (binary), IQ (continuous), and SRS (continuous)

Setting Phenotype Colors

Via CLI (JSON style file)

```
// style.json
{
  "phenotype_colors": {
    "epilepsy": "#e74c3c",
    "IQ": "#3498db",
    "SRS": "#e67e22"
  }
}
```

```
pypedigree family.ped --style style.json -o output.png
```

Via Python

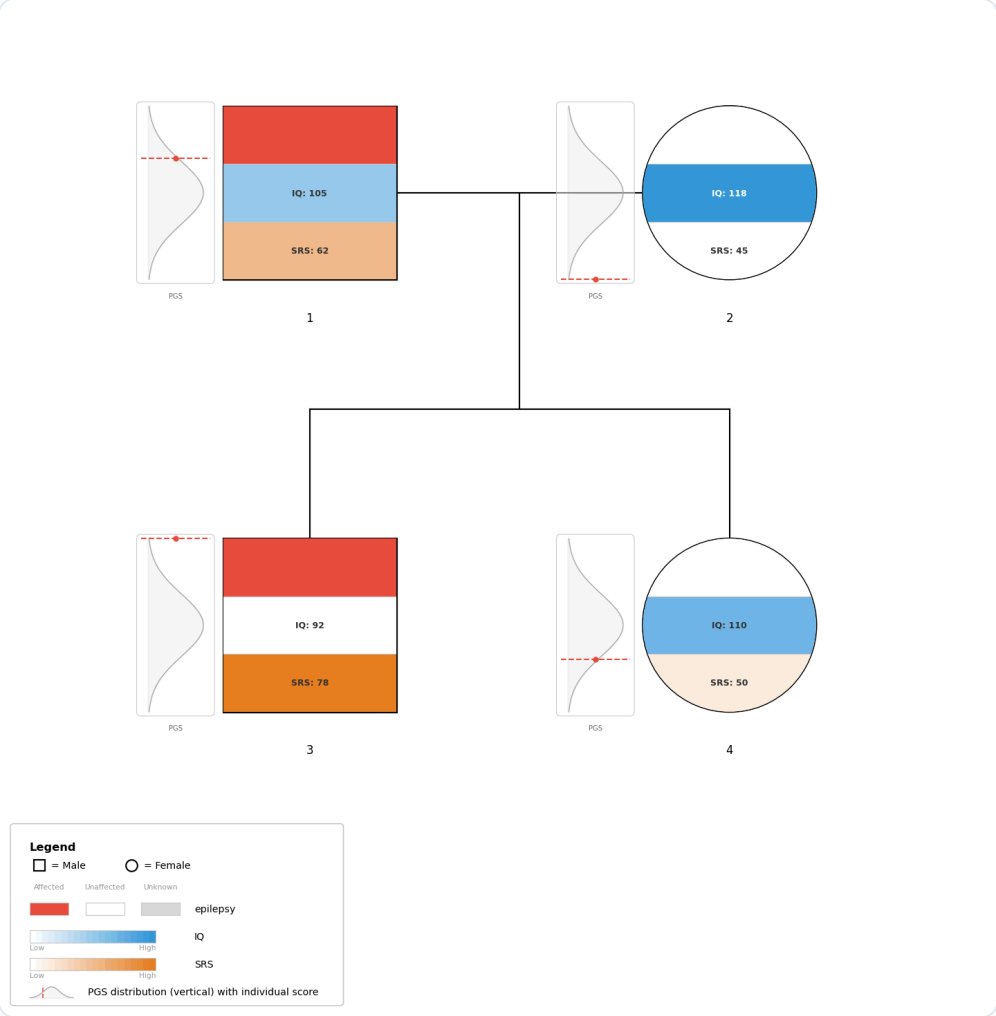
```
style = PedigreeStyle()
style.phenotype_colors = {
    "epilepsy": "#e74c3c",
    "IQ": "#3498db",
    "SRS": "#e67e22",
}
fig = render_pedigree(ped, style=style)
```

Auto-assigned

When no `phenotype_colors` are set, colors are assigned from the default palette. This happens for every entry point — the CLI, `render_pedigree()` and `PedigreeBuilder` — via `resolve_style()`, so all three produce the same figure for the same input.

PGS (Polygenic Score) Display

When a column is named `PGS` (case-insensitive), it is rendered as a vertical Gaussian curve to the left of each individual's shape, instead of a band. The curve's position along the x-axis reflects the PGS value relative to other individuals.



Pedigree with phenotype bands + PGS Gaussian curves

Multiple PGS Columns

Any column starting with `PGS` (case-insensitive) is treated as a PGS column. For example: `PGS_autism`, `PGS_IQ`, `PGS_ADHD`. All PGS columns share a single gaussian curve, with each trait's individual score marked by a colored dashed line. Colors are set via the `pgs_colors` style option.

Custom Gradient Ranges

By default, gradient values are normalized to [0,1] using the data's min/max. You can override this with the `gradient_range` style option to define fixed scales:

```
{
    "gradient_range": {
        "SRS": [40, 90]
    }
}
```

Diverging (Tricolor) Gradient

For phenotypes with a meaningful midpoint (e.g., IQ where 100 is average), use `gradient_diverging` to create a red→white→green scale:

```
{
    "gradient_diverging": {
        "IQ": {"min": 40, "mid": 100, "max": 160, "color_low": "#e74c3c", "color_
    }
}
```

Values below the midpoint blend toward `color_low`, values above blend toward `color_high`, and the midpoint is white.

Categorical Columns

Each distinct value of a `c_` column gets its own color, assigned in the order the values appear in the file so the result is reproducible. The column's own `phenotype_colors` entry is used for the first value; the remaining values avoid both that color and the colors of the other phenotype columns, so two bands on the same individual stay visually distinct. The legend lists exactly the values present in the rendered pedigree.

Default Color Palette

When colors are auto-assigned, the following 8-color palette is used:

#e74c3c	#3498db	#e67e22	#2ecc71	#9b59b6	#1abc9c	#f1c40f	#e91e63

Phenotype Display Modes

By default, pypedigree auto-detects how to render each phenotype column (binary vs. continuous). You can override this behaviour by adding a **prefix** to the column name in the header row. The prefix is stripped from the display name in legends and labels.

Prefix Reference

PREFIX	MODE	RENDERING	USE CASE
`b_`	Binary	Color band when affected, white when unaffected, grey when unknown. No text by default.	Diagnoses, yes/no traits
`g_`	Gradient	White→color continuous gradient scale. Value labels shown inside band.	Quantitative scores (IQ, SRS, etc.)
`o_`	Outline	Colored shape border when affected, instead of a filled band. Does not consume band space.	Proband marker, secondary highlight
`c_`	Categorical	Distinct color per unique string value, shown as a colored band with label.	Diagnosis categories, subtypes
`cr_`	Carrier	Carrier status of one gene or condition: `0` not a carrier, `1` carrier, `2` obligate carrier. Drawn as a patterned section of the symbol, with its own legend row.	CFTR, HEXA, ATM carrier status
`gi_`	`GRADIENT_INVERTED`	Color→white gradient (inverted). High value = white, low value = full	Any numeric	Color gradient bar

PREFIX	MODE	RENDERING	USE CASE	
		color. Use for metrics where low = bad (e.g., IQ: low IQ = red).		(reversed direction)
`a_`	*Annotation*	Text annotation displayed below the individual's ID label. Not rendered as a phenotype band.	Any text string	Italic grey text below ID
(none)	Auto	Resolved once per column from all of its values: categorical if any value is a string, gradient if any numeric value falls outside 0/1/2, binary otherwise. A column therefore never mixes band types, and its legend row always matches its bands.	Backward-compatible default	

Example Header with Prefixes

```
FID  ID  Father  Mother  Sex  b_ASD  g_IQ  o_Proband  c_Diagnosis
```

This defines four phenotype columns:

- **ASD** — binary mode (filled/empty/grey bands)
- **IQ** — gradient mode (white-to-color scale with value label)
- **Proband** — outline mode (colored border when value is 1/2)
- **Diagnosis** — categorical mode (unique color per string value)

> **Prefix is stripped from display name**
>
> The prefix (`b_` , `g_` , `o_` , `c_`) is removed from the phenotype name shown in legends and labels. For example, `b_ASD` appears as "ASD" in the legend.

Complete Example File

```
FID   ID        Father  Mother  Sex  b_ASD  g_IQ  o_Proband  c_Diagnosis
FAM1  father    0       0       1    0      105   0          Control
FAM1  mother    0       0       2    0      98    0          Control
FAM1  child1    father  mother  1    2      72    1          ASD
FAM1  child2    father  mother  2    1      110   0          Control
```

Python API

The `PhenotypeDisplayMode` enum is exported from pypedigree and represents the five display modes:

```python
from pypedigree import PhenotypeDisplayMode

# Available modes:
PhenotypeDisplayMode.AUTO         # No prefix, auto-detect
PhenotypeDisplayMode.BINARY       # b_ prefix
PhenotypeDisplayMode.GRADIENT     # g_ prefix
PhenotypeDisplayMode.OUTLINE      # o_ prefix
PhenotypeDisplayMode.CATEGORICAL  # c_ prefix
```

Combining with smart header mapping

Prefixed column names work seamlessly with the smart header-based column mapping. Any column whose name (after alias matching) is not a known structural column is treated as a phenotype, and the prefix determines its display mode.

Annotations

Annotations are text lines displayed below each individual's ID label. They are useful for showing variant names, gene names, or other per-individual notes.

Inline Annotations (a_ prefix)

Add columns with the `a_` prefix in the .ped header. The prefix is stripped for display; only the value is shown.

```
FID  IID  PAT  MAT  SEX  b_Autism  IQ  g_SRS  a_Gene1    a_Gene2
196  001  002  003  1    2         90  90     DMXL2
196  003  0    0    2    1         75  -9     DMXL2      CAMK2B
196  006  002  003  1    2         70  87                CAMK2B
```

Missing values (`0` , `-9` , `NA` , empty) produce no annotation line. When annotations are present, the vertical spacing between generations is automatically increased to prevent overlap.

External Annotation File

Annotations can also be loaded from a separate TSV file via `--annotations PATH` :

```
IID    Gene1    Gene2
001    DMXL2
003    DMXL2    CAMK2B
006             CAMK2B
```

Python API

```
from pypedigree import PedigreeBuilder

builder = PedigreeBuilder("FAM1")
builder.add_individual("001", sex="male",
                       annotations=["DMXL2"])
builder.add_individual("003", sex="female",
                       annotations=["DMXL2", "CAMK2B"])
```

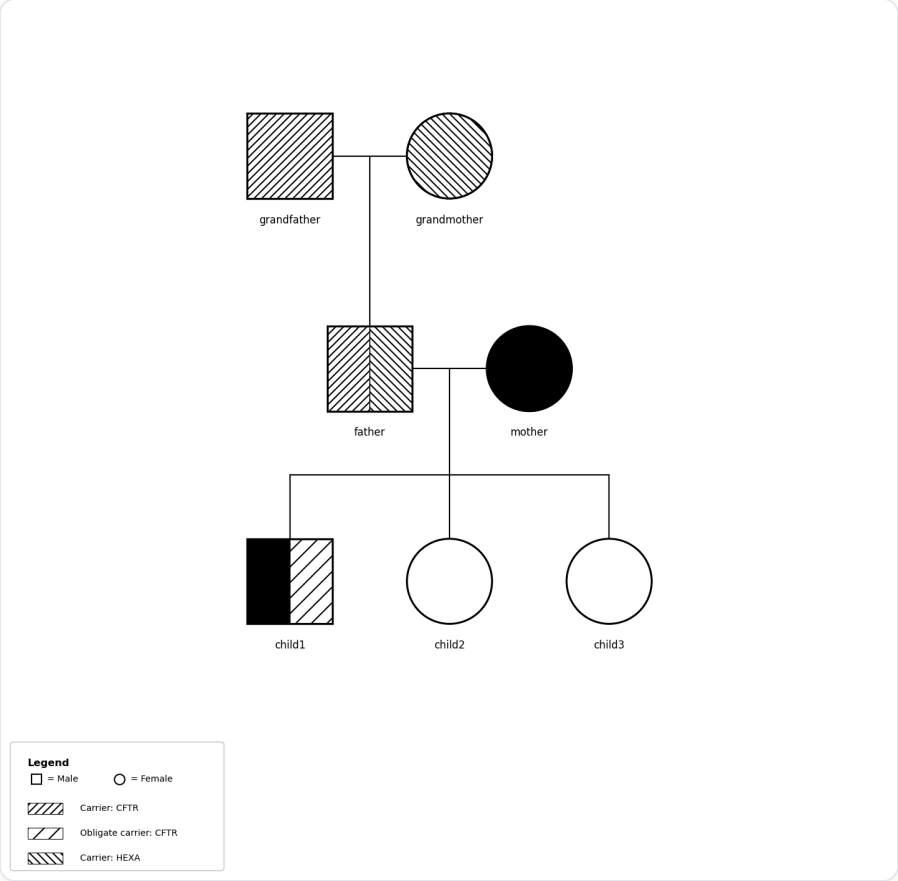
Carrier Notation

pypedigree follows the NSGC focused revision of the standardized pedigree nomenclature (Bennett et al., *J Genet Couns* 2022;31(6):1238–48), which no longer marks carriers with a dot: the symbol is divided into as many sections as needed, and each carrier result gets its own fill pattern, defined in the legend. An affected carrier, or the carrier of two conditions, therefore shows every result.

Visual Representations

STATUS	SYMBOL (DEFAULT, NSGC 2022)	SYMBOL (`CARRIER_NOTATION: "LEGACY"`)
Carrier	Patterned section, one pattern per carrier source	Half-filled shape
Obligate carrier	The same pattern at half the density	Center dot
Affected carrier	Symbol split in two: filled section, then patterned section	Half-filled shape (the affected fill is not shown)

Sections are vertical and ordered left to right: the affected section first, then one per carrier source in column order. A carrier of a single result has a fully patterned symbol. When phenotype bands are drawn, each carrier source is an extra band below them, at the same place for everyone.



`carrier_genes.ped`: one `cr_` column per gene. The father carries both CFTR and HEXA; child1 is affected and an obligate CFTR carrier.

File Format

Carrier status comes from the `carrier` column and from any number of `cr_` columns (one per gene or condition, e.g. `cr_CFTR`). Each is a separate carrier source, with its own pattern and legend row, and accepts:

VALUE	MEANING
`0`	Not a carrier
`1`	Carrier (heterozygous)
`2`	Obligate carrier, as declared in the file (pypedigree does not infer it)
`-9`, `NA`, empty	Unknown (`cr_` columns only): no section, or an “NA” band

Example with carrier columns

```
family_id  sample_id  father  mother  sex  pheno  cr_CFTR  cr_HEXA
FAM1       father     0       0       1    1      1        1
FAM1       mother     0       0       2    2      0        0
FAM1       son        father  mother  1    2      2        0
FAM1       daughter   father  mother  2    1      0        -9
```

Programmatic API (PedigreeBuilder)

Use the `carrier` and `obligate_carrier` parameters of `add_individual()` for the `carrier` column, and `cr_`-prefixed keys in `phenotypes` for per-gene status:

```python
from pypedigree import PedigreeBuilder

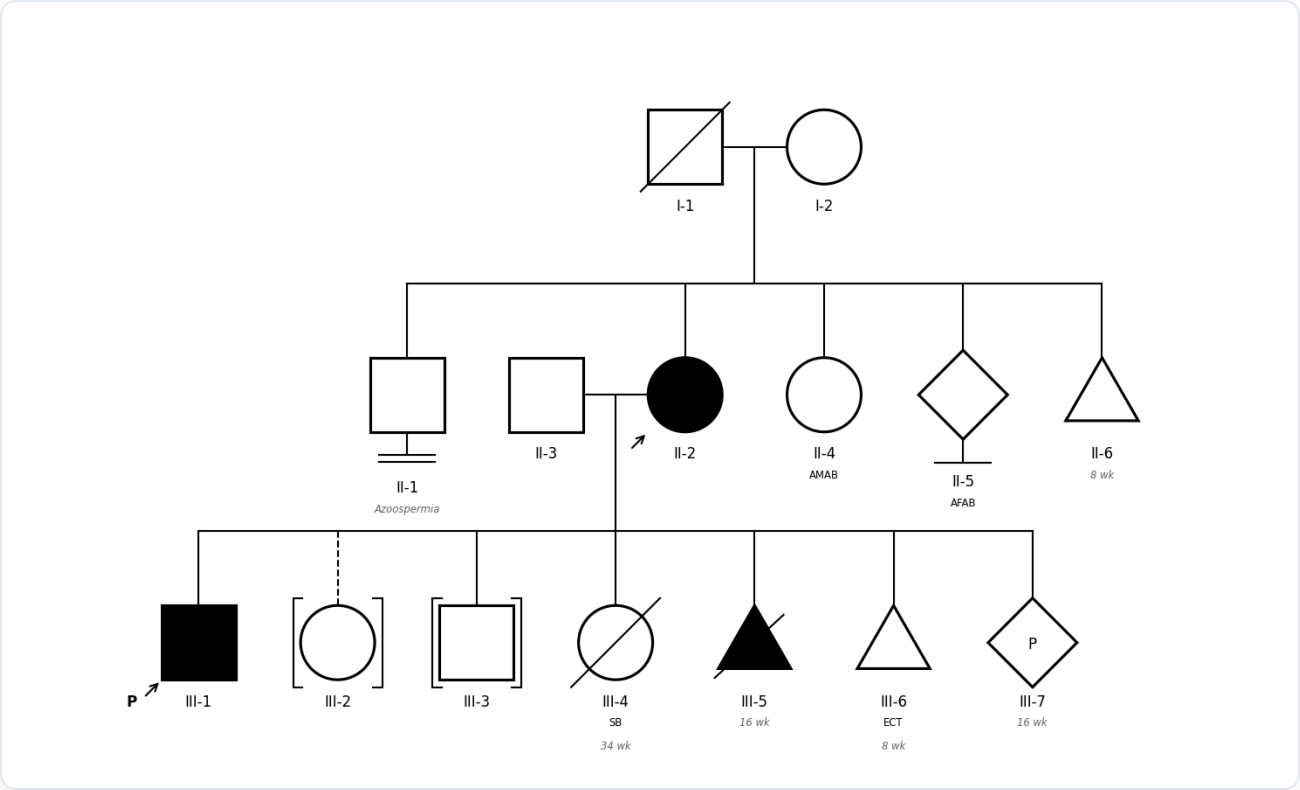
builder = PedigreeBuilder("FAM1")
builder.add_individual("father", sex="male", carrier=True)
builder.add_individual("mother", sex="female", phenotypes={"cr_CFTR": 1})
builder.add_individual("child", sex="female",
    father="father", mother="mother",
    obligate_carrier=True)
fig = builder.render(output="carriers.png")
```

Legacy notation

The pre-2022 notation stays available for the `carrier` column: `PedigreeStyle(carrier_notation="legacy")`, `"carrier_notation": "legacy"` in a style file, or `--carrier-notation legacy` on the command line. `cr_` columns are still drawn as patterned sections. The radius of the legacy center dot is set by `carrier_dot_radius` (default `0.08`).

Standard Notation

Beyond sex, affection status, death, twins and carriers, pypedigree draws the other symbols of the NSGC nomenclature (Bennett et al., *J Genet Couns* 2008;17:424–33, and its 2022 revision centred on sex and gender) in the PNG and the interactive SVG, in both layouts. Each comes from one column, recognised by name in the header row.



`nomenclature.ped` *(family FAM1): infertility and childlessness by choice (II-1, II-5), the consultand (II-2), a transgender woman and a non-binary person (II-4, II-5), a miscarriage (II-6); children adopted in and out (III-2, III-3), a stillbirth, an affected termination, an ectopic pregnancy and an ongoing pregnancy (III-4 to III-7).*

Columns and codes

Column names and codes are case-insensitive (aliases in brackets). `0`, `-9`, `NA`, `.` or an empty cell mean nothing to note; any other value is an error naming the line.

COLUMN	CODES	DRAWN AS
`pregnancy` (`pregnancy_outcome`, `outcome`)	`P` ongoing, `SAB` miscarriage, `TOP`	`P`: “P” inside the symbol. `SAB`, `TOP`, `ECT`: a triangle, filled when affected, struck through for a

COLUMN	CODES	DRAWN AS
	termination, `ECT` ectopic, `SB` stillbirth	termination, “ECT” below. `SB`: the symbol struck through, “SB” below
`adopted` (`adoption`)	`in` / `1` (parents given are adoptive), `out` / `2` (they are biological)	Brackets around the symbol; the line to adoptive parents is dashed
`childless` (`childlessness` , `infertility`)	`choice` / `1` , `infertility` / `2`	A line below the symbol ending in one bar (no children by choice or reason unknown), or two (infertility)
`art` (`assisted_reproduction`)	`D` gamete donor, `S` surrogate (`GC` accepted)	“D” or “S” inside the symbol
`gender_identity` (`identity`)	`man` , `woman` , `nonbinary` (also `M` / `F` / `NB` , `1` / `2` / `3`)	The symbol follows the gender identity (square, circle, diamond); `sex` becomes the sex assigned at birth, written below as AMAB or AFAB when the two differ
`consultand`	`1`	An arrow without “P”

> **The `gender` column**
>
> A column named `gender` is the sex column, as before, unless the header also has a `sex` column: then it is the gender identity.

Gestational age, karyotype or the reason for infertility go in `a_` annotation columns. Notes (“SB”, “ECT”, “AMAB”) are written under the ID label, before annotations. A pregnancy-loss triangle has no room for phenotype bands: in a figure with bands it takes the color of the individual's first band. On a symbol with bands, which carry text of their own, the letter “P”, “D” or “S” moves beside the upper-right corner.

Programmatic API

```
builder.add_individual("c1", father="f", mother="m", pregnancy="SAB", annotations
builder.add_individual("c2", sex="male", father="f", mother="m", gender="woman",
```

`build_notation()` turns the same codes into a `Notation`, the frozen dataclass attached to every `Individual` and `NodeLayout`.

Not covered

The diagonal lines that join a gamete donor or a surrogate to a pregnancy in the NSGC figures: in a pedigree file a donor is a parent and is joined to the other parent like any couple. Several individuals in one symbol (“n”), documented evaluation (“*”), the asymptomatic-carrier vertical line, and childlessness of a couple (the marker hangs from one individual).

Interactive SVG Export

pypedigree can generate interactive SVG files with built-in tooltips, click-to-highlight, and zoom/pan capabilities. Interactive SVGs are self-contained HTML/SVG files that work in any modern browser with **no additional dependencies**.

Features

- **Tooltips on hover** — Hovering over an individual displays their sample ID, sex, phenotype, and other metadata.
- **Click to highlight relatives** — Clicking an individual highlights their parents, children, and partners.
- **Zoom and pan** — Mouse wheel to zoom, Ctrl+drag (or middle-button drag) to pan around large pedigrees.
- **No extra dependencies** — All interactivity is embedded as inline JavaScript within the SVG file.

Same figure as the PNG

The interactive SVG draws everything the PNG/PDF output draws: phenotype bands (binary, gradient, diverging, categorical), `o_` outline colors, PGS distribution curves, `a_` annotations, carrier notation and the legend (hidden with `--no-legend`). Both renderers take their decisions from the same code (`pypedigree.phenotypes` and `pypedigree.carriers`), so the two outputs always agree. Several families are drawn side by side, each with its title.

Each individual's tooltip also lists every value, for example `ID: 1 | Sex: Male | Phenotype: Affected | IQ: 90 | Dx: ASD`. The circular layout (`--circular`) is not available in interactive mode.

CLI Usage

Use the `--interactive` flag to generate an interactive SVG instead of a static image:

```
pypedigree family.ped --interactive -o pedigree.svg
```

> **Output format**
>
> The `--interactive` flag automatically produces SVG output. If the output path has a different extension, it will be overridden to `.svg`.

Python API

`render_interactive_svg(pedigree, ...)`

Render all families in a pedigree as an interactive SVG. Returns the SVG markup as a string.

```
from pypedigree import parse_pedigree, render_interactive_svg

ped = parse_pedigree("family.ped")
svg_str = render_interactive_svg(ped, output="interactive.svg")
```

`PedigreeBuilder.render_interactive_svg(output=None, *, style=None, show_labels=True, show_legend=True)`

Build and render as an interactive SVG from the builder API:

```
from pypedigree import PedigreeBuilder

builder = PedigreeBuilder("FAM1")
builder.add_individual("father", sex="male")
builder.add_individual("mother", sex="female")
builder.add_individual("child", sex="male",
    father="father", mother="mother",
    phenotype="affected")
svg_str = builder.render_interactive_svg(output="interactive.svg")
```

Styling & Configuration

All visual parameters are controlled through the `PedigreeStyle` dataclass. Styles can be set in Python or loaded from a JSON file via `--style`.

PedigreeStyle Parameters

PARAMETER	TYPE	DEFAULT	DESCRIPTION
`shape_size`	float	0.8	Individual shape side length
`shape_line_width`	float	1.5	Shape outline width
`fill_color`	str	"black"	Affected individual fill color
`outline_color`	str	"black"	Shape and marker outline color
`phantom_line_style`	str	"--"	Line style for phantom parents
`line_color`	str	"black"	Connection line color
`line_width`	float	1.0	Connection line width
`double_line_gap`	float	0.06	Gap between consanguinity double lines
`broken_gap_size`	float	0.15	Gap size in broken relationship lines
`font_size`	float	8.0	Label and text font size, in points. The legend panel is sized from it (text, swatches and row height), so raise it to keep a figure legible once scaled down for print
`font_family`	str	"sans-serif"	Font family for all text
`label_offset`	float	0.15	Distance below shape for label
`generation_gap`	float	2.0	Vertical spacing between generations

PARAMETER	TYPE	DEFAULT	DESCRIPTION
`sibling_gap`	float	1.5	Horizontal spacing between siblings
`couple_gap`	float	1.5	Horizontal spacing between partners
`min_node_distance`	float	1.2	Minimum horizontal distance between any two nodes
`phenotype_colors`	dict	{}	Phenotype name → hex color mapping
`dpi`	int	150	Output resolution
`fig_width`	float	10.0	Default figure width (inches)
`fig_height`	float	8.0	Default figure height (inches)
`margin`	float	0.5	Figure margin (inches)
`carrier_notation`	str	"nsgc2022"	`"nsgc2022"` : patterned sections, one per carrier result (Bennett et al. 2022). `"legacy"` : half-filled shape and center dot
`carrier_dot_radius`	float	0.08	Radius of the center dot for obligate carriers (legacy notation)
`show_phenotype_labels`	bool	False	Show YES/NO text inside binary phenotype bands
`pgs_colors`	dict	{}	Map of PGS column name to hex color for marker line (e.g. `{"PGS_autism": "#3498db"}`)
`gradient_range`	dict	{}	Custom min/max for gradient normalization (e.g. `{"SRS": [40, 90]}`)
`gradient_diverging`	dict	{}	Tricolor diverging gradient config (e.g. `{"IQ": {"min": 40, "mid": 100, "max": 160, "color_low": "#e74c3c", "color_high": "#2ecc71"}}`)

PARAMETER	TYPE	DEFAULT	DESCRIPTION
`zero_pad_ids`	int	0	Zero-pad individual IDs to this width (0 = no padding, 3 = "001")
`radial_gap`	float	2.5	Radial spacing between concentric rings (circular layout)
`base_radius`	float	3.0	Radius of first generation ring (circular layout)

JSON Style File Example

```
{
  "shape_size": 1.0,
  "font_size": 10,
  "outline_color": "#888888",
  "generation_gap": 2.5,
  "dpi": 300,
  "phenotype_colors": {
    "epilepsy": "#e74c3c",
    "IQ": "#3498db"
  },
  "pgs_colors": {
    "PGS_autism": "#3498db",
    "PGS_IQ": "#e74c3c",
    "PGS_ADHD": "#e67e22"
  },
  "gradient_range": {
    "SRS": [40, 90]
  },
  "gradient_diverging": {
    "IQ": {
      "min": 40, "mid": 100, "max": 160,
      "color_low": "#e74c3c", "color_high": "#2ecc71"
    }
  },
  "zero_pad_ids": 3
}
```

Auto-Scaling

For large pedigrees, the figure size is automatically scaled based on the layout bounds. The `fig_width` and `fig_height` values serve as minimums; actual dimensions grow with the number of individuals and generations.

PLINK Integration

pypedigree provides native support for PLINK bioinformatics file formats, enabling seamless interoperability with tools like PLINK, KING, GCTA, and other genetics software.

Reading .fam Files

PLINK `.fam` files use the standard 6-column pedigree format (FID, IID, PAT, MAT, SEX, PHENO), whitespace-delimited, with no header row. Files with the `.fam` extension are auto-detected by the CLI.

```
# Python API
from pypedigree import parse_fam_file, render_pedigree

ped = parse_fam_file("family.fam")
render_pedigree(ped, output="family.png")

# CLI — .fam extension is auto-detected
# pypedigree family.fam -o family.png
```

Reading PLINK .ped Files

PLINK `.ped` files have 6 standard columns followed by genotype data (pairs of alleles). Use `parse_plink_ped()` or the `--plink` CLI flag to read only the pedigree columns and ignore genotype data.

```
# Python API
from pypedigree import parse_plink_ped

ped = parse_plink_ped("family.ped")

# CLI — use --plink flag
# pypedigree family.ped --plink -o family.png
```

Exporting to .fam

Export a `Pedigree` to PLINK `.fam` format for use with external tools. Internal constructs (phantom parents, duplicate nodes) are automatically excluded, and references are resolved back to original IDs.

```
from pypedigree import parse_pedigree, export_fam

ped = parse_pedigree("family.ped")
export_fam(ped, "output.fam")  # Ready for PLINK/KING
```

API Reference

FUNCTION	DESCRIPTION
`parse_fam_file(path)`	Parse a PLINK .fam file (6 columns, no header)
`parse_plink_ped(path)`	Parse a PLINK .ped file (ignores genotype columns)
`export_fam(pedigree, path)`	Export to .fam format (tab-delimited, 6 columns)

Layout Engine

The layout engine computes (x, y) coordinates for all individuals and connection lines. It has **no matplotlib dependency** and produces pure numeric `FamilyLayout` objects.

Pipeline

1. **Assign generations** — BFS from founders, children one generation below parents. Partners aligned to same generation, then propagated until every child sits below both parents. A duplicate node stands on the generation of the individual it repeats. Uses a parent→child adjacency index.
2. **Compute horizontal positions** — Each union's descendants form a block. Sibling blocks are packed left to right against each other's contours, generation by generation, and every couple is centred over its children. A married twin's partner stands on the outer side of the twin group, so no spouse is drawn between twins.
3. **Break tangled loops** — When both partners descend from people in the pedigree (consanguinity, marriages between branches), they are drawn side by side and the second partner's sibship line stretches to reach them. Siblings are ordered so that the two branches of a loop meet, which keeps that line clear in most pedigrees. Where it would still overlap another sibship, the partner is drawn twice instead: among their siblings, and as a duplicate next to their partner, joined by a dotted arc, as the nomenclature allows.
4. **Build geometry** — Compute node positions, connection edges (relationship, descent, sibship, individual lines), twin connectors, and duplicate arcs. Every line is horizontal or vertical, as the standard nomenclature requires: a single child that is not centred under its parents (possible only in a loop) hangs from a short sibship-level segment rather than a diagonal.

> **Guarantees**
>
> Nodes on a row stay at least `sibling_gap` apart (`couple_gap` between partners), and the sibship lines of two different couples never overlap, so every child can be traced to its parents. The test suite checks both on every fixture and on generated pedigrees of up to 200 individuals, with and without loops. The same rules apply at every size.

Performance

PEDIGREE SIZE	LAYOUT TIME	NOTES
9 individuals	< 1ms	Typical clinical pedigree

PEDIGREE SIZE	LAYOUT TIME	NOTES
45 individuals	< 1ms	Extended family
100 individuals	≈ 1ms	Including loop breaking
200 individuals	≈ 2ms	Large research pedigree
500 individuals	≈ 5ms	Densely interbred, many loops

Pedigree Symbols

SYMBOL	MEANING
Square ☐	Male
Circle ○	Female
Diamond ◇	Unknown sex
Filled shape	Affected
Empty shape	Unaffected
Diagonal line through shape	Deceased
Arrow below shape	Proband (index case)
Double horizontal line	Consanguineous union
Broken horizontal line	Dissolved/broken union
Patterned section	Carrier, one pattern per carrier source (NSGC 2022)
Sparser pattern	Obligate carrier (NSGC 2022)
Half-filled shape / center dot	Carrier / obligate carrier (legacy notation)
Dashed shape outline	Phantom (inferred) parent
Horizontal bar between twins	Monozygotic twins

SYMBOL	MEANING
V-lines from descent line	Dizygotic twins

Data Models

Enums

`Sex`

`Sex.MALE = 1`
: Male individual (rendered as square).

`Sex.FEMALE = 2`
: Female individual (rendered as circle).

`Sex.UNKNOWN = 0`
: Unknown sex (rendered as diamond).

`Phenotype`

`Phenotype.UNAFFECTED = 1`
: Unaffected (empty shape).

`Phenotype.AFFECTED = 2`
: Affected (filled shape).

`Phenotype.UNKNOWN = 0`
: Unknown status.

`PhenotypeDisplayMode`

`PhenotypeDisplayMode.AUTO = "auto"`
: Auto-detect from data (default, no prefix).

`PhenotypeDisplayMode.BINARY = "binary"`
: Binary rendering: color/white/grey bands (`b_` prefix).

`PhenotypeDisplayMode.GRADIENT = "gradient"`
: Continuous gradient scale with value labels (`g_` prefix).

`PhenotypeDisplayMode.OUTLINE = "outline"`
: Colored shape border when affected (`o_` prefix).

`PhenotypeDisplayMode.CATEGORICAL = "categorical"`
: Distinct color per unique string value (`c_` prefix).

Notation enums

`Pregnancy`
: `ONGOING ("P")`, `MISCARRIAGE ("SAB")`, `TERMINATION ("TOP")`, `ECTOPIC ("ECT")`, `STILLBIRTH ("SB")`.

`Adoption`
: `IN`, `OUT`.

`Childlessness`
: `BY_CHOICE`, `INFERTILITY`.

`ReproductiveRole`
: `DONOR ("D")`, `SURROGATE ("S")`.

`Gender`
: `MAN`, `WOMAN`, `NONBINARY`: decides the symbol.

They are grouped in `Notation` (`pregnancy`, `adoption`, `childlessness`, `reproductive_role`, `gender`, `is_consultand`), the `notation` field of `Individual` and `NodeLayout`. `NodeLayout` also carries `notes` (text written under the label) and `marker_depth` (room taken by a childlessness marker); `EdgeType.CHILDLESS` marks that marker's segments.

`TwinType`

`TwinType.MONOZYGOTIC = "MZ"`
: Identical twins (horizontal bar connector).

`TwinType.DIZYGOTIC = "DZ"`
: Fraternal twins (plain V-lines).

`TwinType.UNKNOWN = "TW"`
: Unknown zygosity (? marker).

Core Classes

`Individual`

Represents one person in the pedigree.

`sample_id: str`
: Unique identifier.

`family_id: str`
: Family this individual belongs to.

`sex: Sex`
: Biological sex.

`phenotype: Phenotype`
: Affected/unaffected status.

`father_id: str | None`
: Father's sample_id (None if unknown).

`mother_id: str | None`
: Mother's sample_id (None if unknown).

`is_deceased: bool`
: Deceased marker.

`is_proband: bool`
: Proband/index case marker.

`is_phantom: bool`
: Inferred parent (dashed outline).

`is_carrier: bool`
: Carrier (heterozygous), from the `carrier` column.

`is_obligate_carrier: bool`
: Obligate carrier, from the `carrier` column.

`twin_info: list[TwinInfo]`
: Twin relationships.

`extra_phenotypes: dict[str, float | None]`
: Normalized extra phenotype values [0, 1].

`extra_phenotypes_raw: dict[str, float | None]`
: Raw (original) extra phenotype values.

`is_founder: bool (property)`
: True if no known parents.

`Union`

Represents a mating relationship between two individuals.

`partner_ids: tuple[str, str]`
: The two partner sample IDs.

`children_ids: list[str]`
: Children from this union.

`is_consanguinous: bool`
: Consanguineous relationship (double line).

`is_broken: bool`
: Dissolved relationship (broken line).

`Family`

A collection of related individuals and their unions.

`family_id: str`
: Family identifier.

`individuals: dict[str, Individual]`
: Map of sample_id → Individual.

`unions: list[Union]`
: All unions in this family.

`get_founders() → list[Individual]`
: Return individuals with no known parents.

`get_children(parent_id) → list[Individual]`
: Return all children of a parent.

`get_union_for_parents(father_id, mother_id) → Union | None`
: Find the union for a parent pair.

`get_unions_for_individual(sample_id) → list[Union]`
: Get all unions involving an individual.

`Pedigree`

Top-level container holding one or more families.

`families: dict[str, Family]`
: Map of family_id → Family.

`phenotype_names: list[str]`
: Ordered list of extra phenotype names.

`family_ids: list[str] (property)`
: List of all family IDs.

`get_family(family_id) → Family | None`
: Get a family by ID.

Layout Classes

`FamilyLayout`

Complete layout result for one family.

`family_id: str`
: Family identifier.

`nodes: list[NodeLayout]`
: Positioned individuals.

`edges: list[EdgeLayout]`
: Positioned connection lines.

`twin_connectors: list[TwinConnector]`
: Twin connection geometry.

`duplicate_arcs: list[DuplicateArc]`
: Arcs connecting duplicate nodes.

`NodeLayout`

`individual_id: str`

Sample ID.

`x: float, y: float`

Position coordinates.

`sex: Sex`

For shape selection.

`phenotype: Phenotype`

For fill.

`label: str`

Display text.

`EdgeLayout`

`start: tuple[float, float]`

Start point (x, y).

`end: tuple[float, float]`

End point (x, y).

`edge_type: EdgeType`

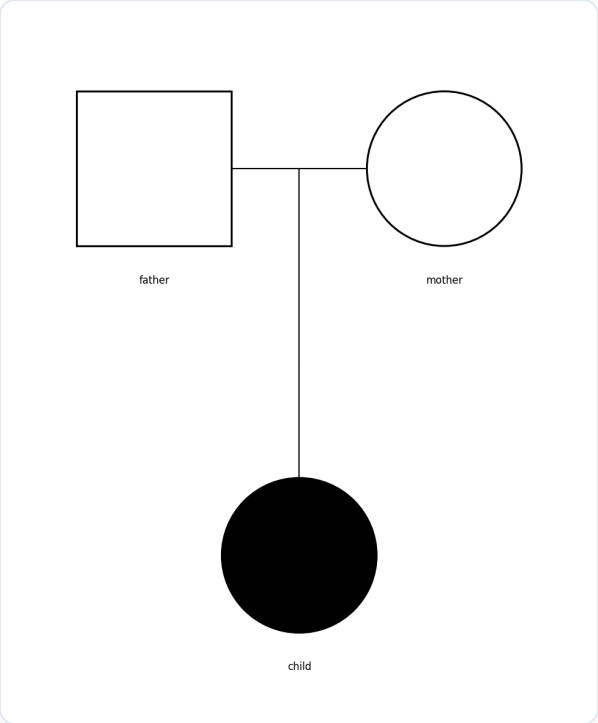
Type: relationship, descent, sibship, or individual.

`line_style: LineStyle`

Style: solid, double, broken, or dashed.

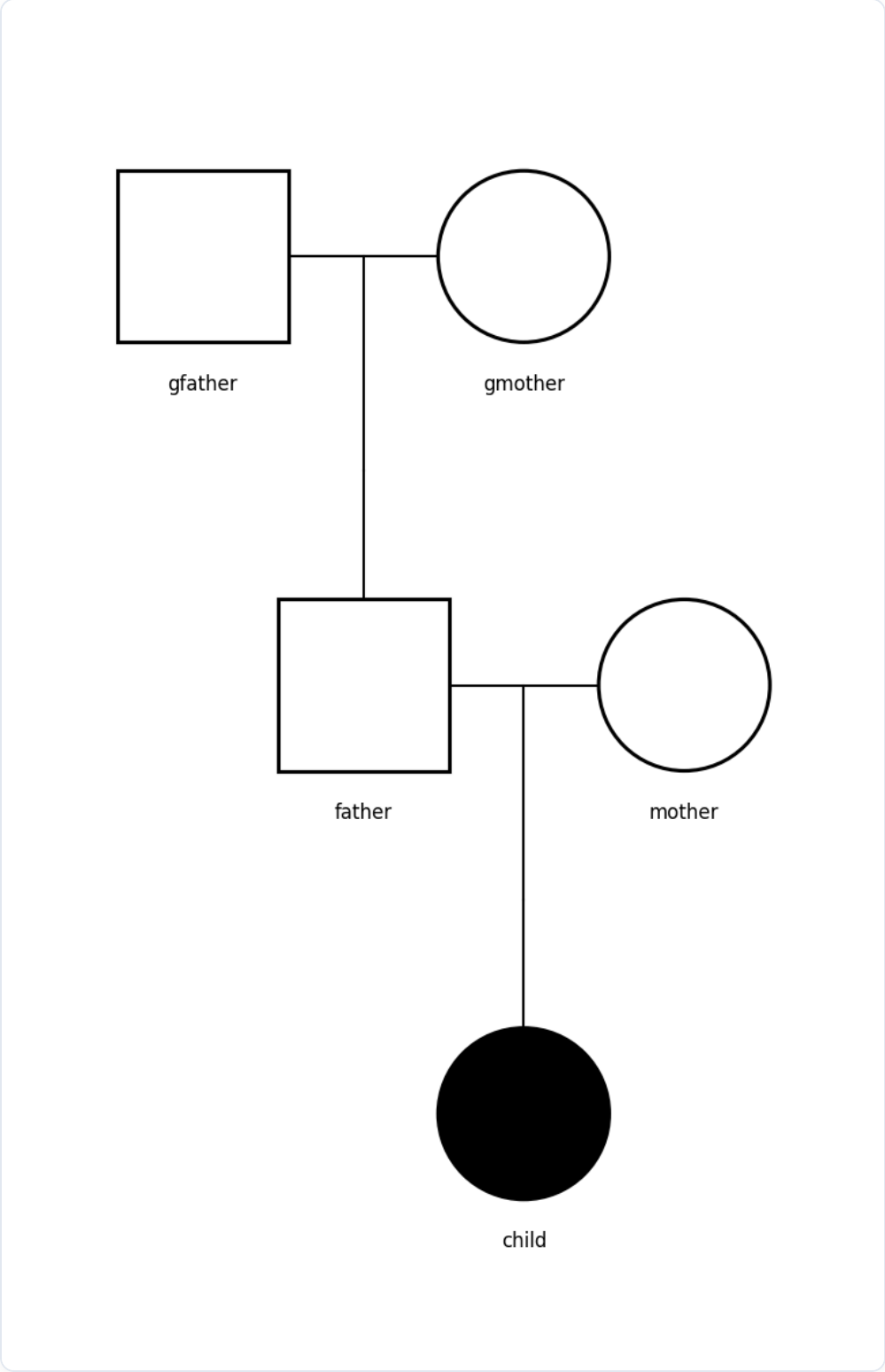
Gallery

Simple Trio



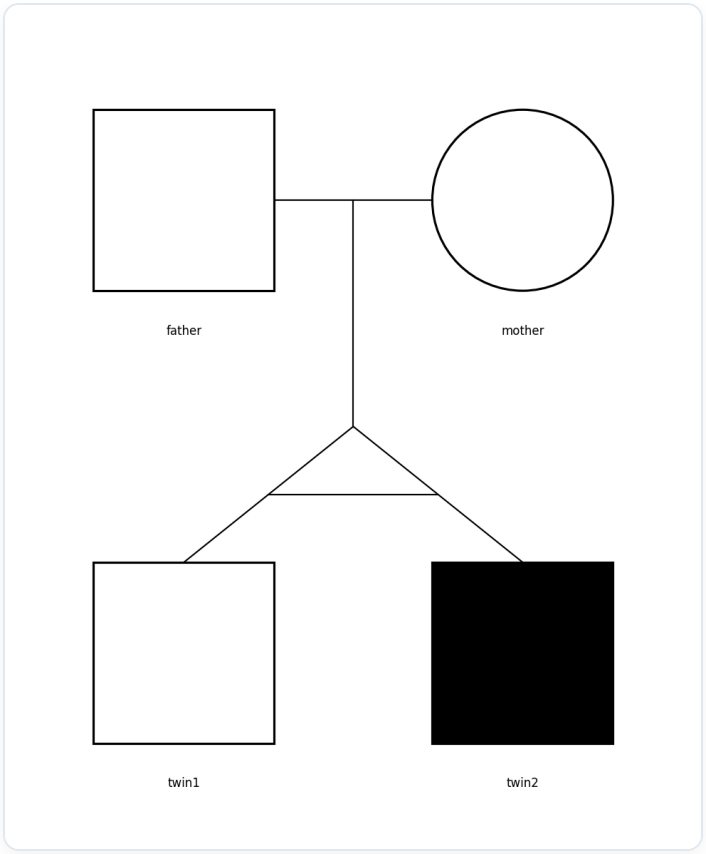
Father, mother, and affected child

Three Generations



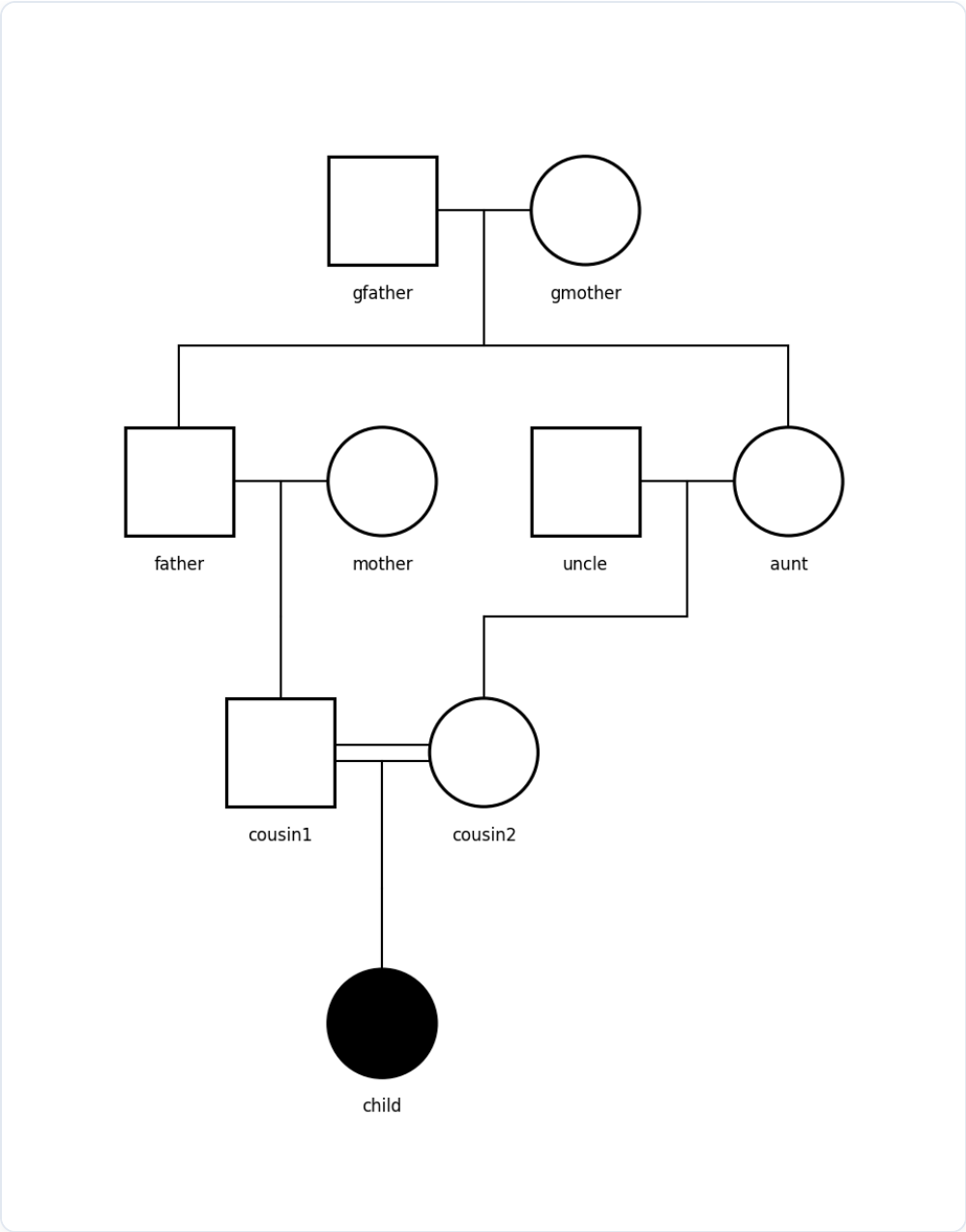
Grandparents, parents, and children across three generations

Monozygotic Twins



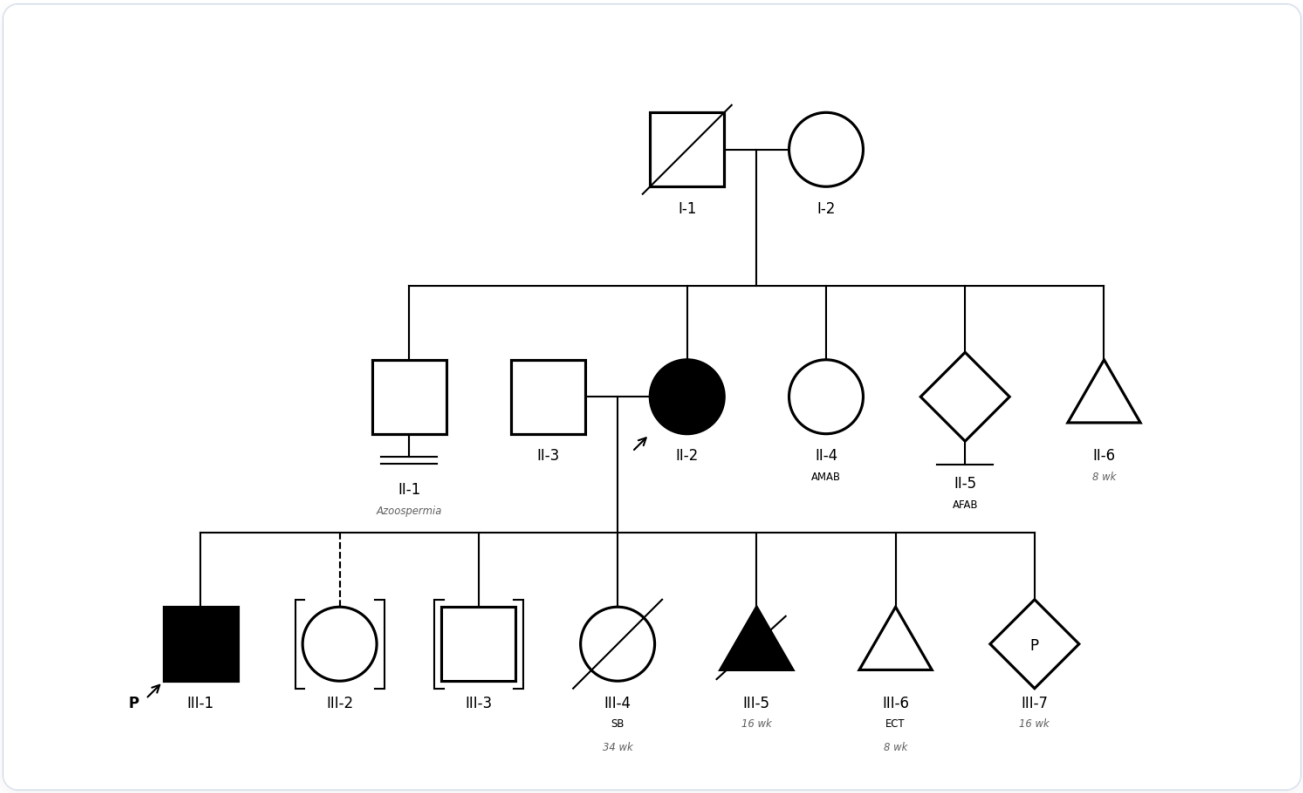
Monozygotic (identical) twins connected by a horizontal bar

Consanguinity



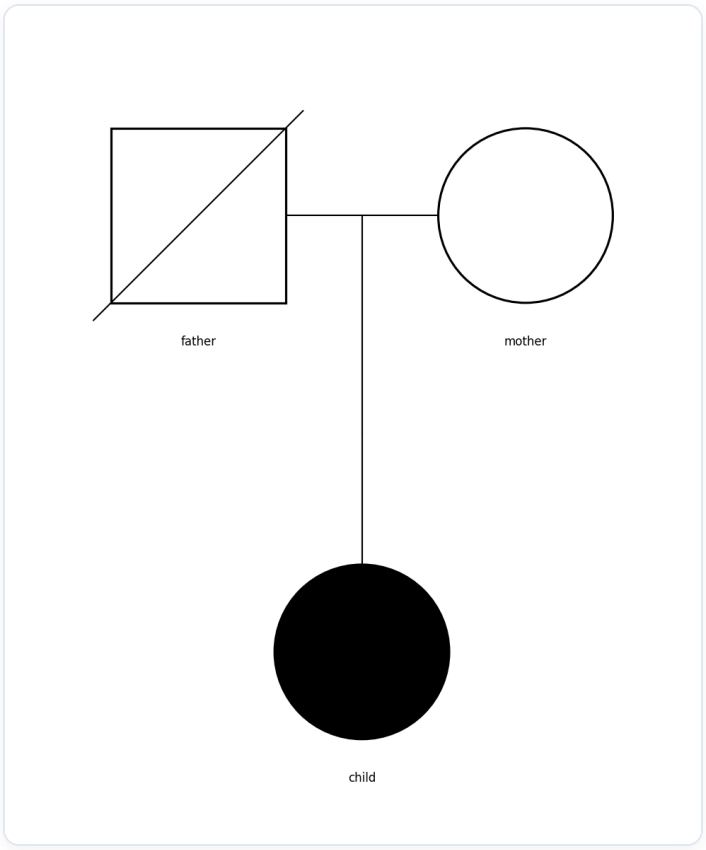
First-cousin marriage shown with double relationship line

Standard Notation



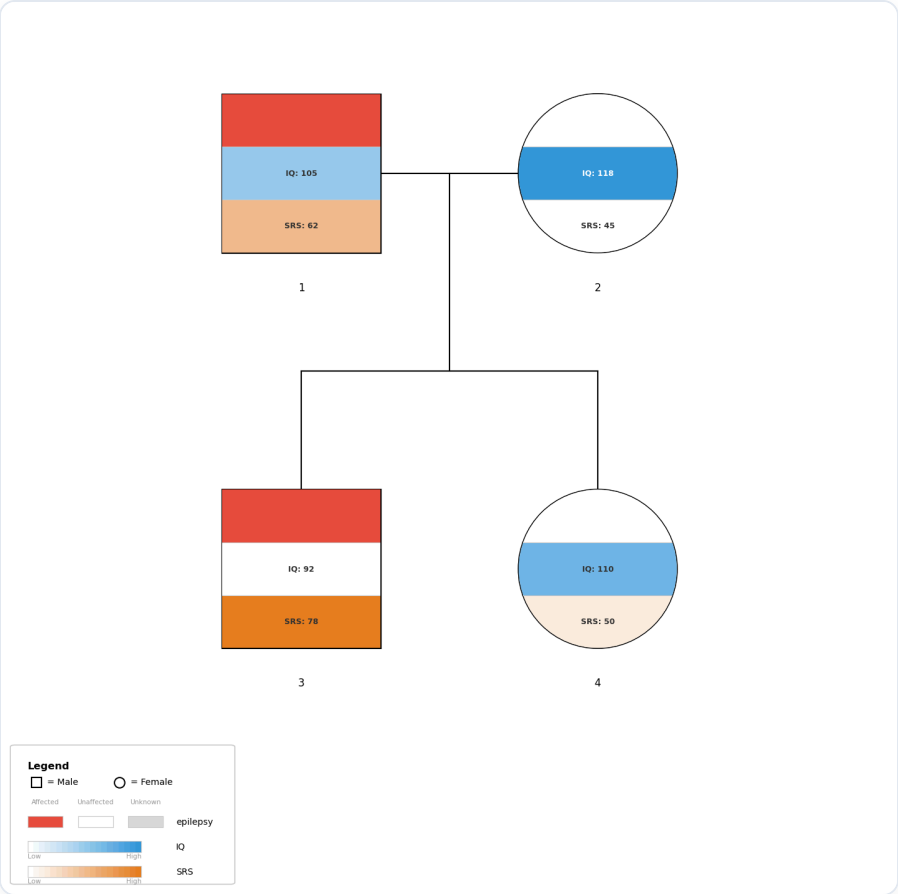
Pregnancies and losses, adoption, childlessness, gender identity and the consultand (`nomenclature.ped`)

Deceased Individuals



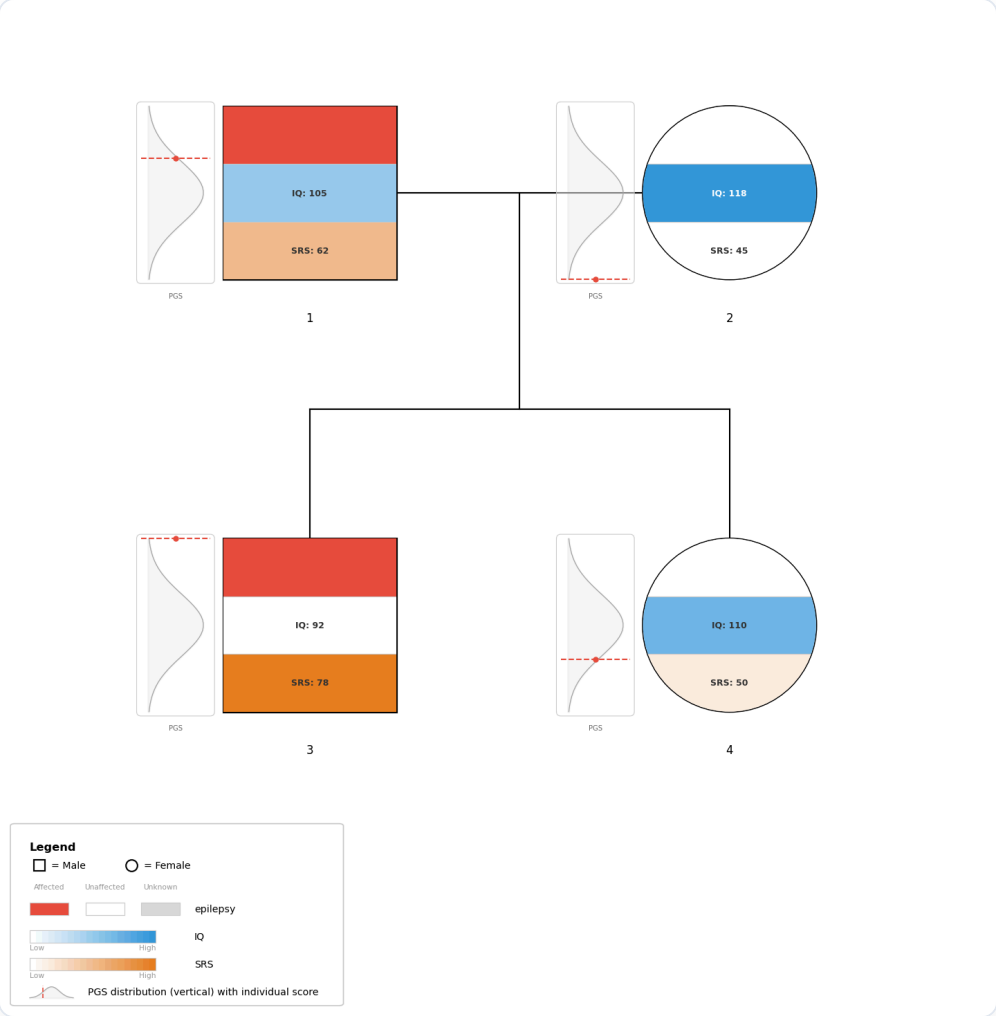
Deceased individuals marked with diagonal line through shape

Multi-Phenotype with Bands



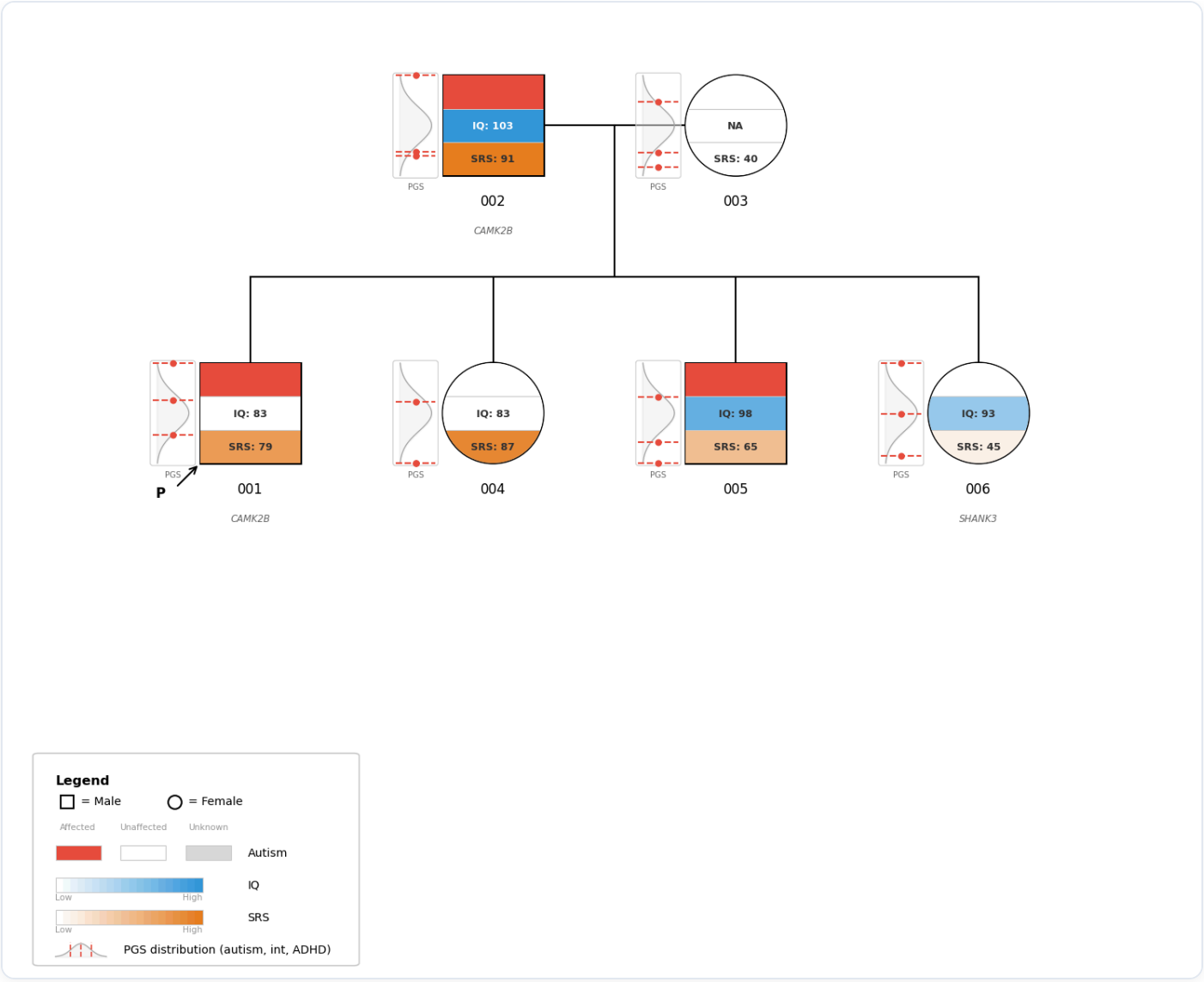
Three phenotypes displayed as colored bands within each shape

Multi-Phenotype with PGS



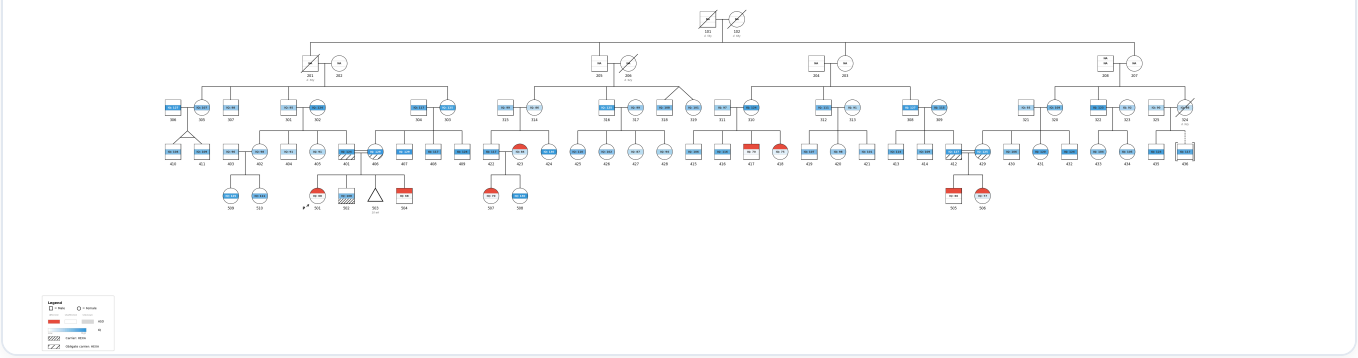
Phenotype bands combined with PGS Gaussian distribution curves

Annotations with Multi-PGS



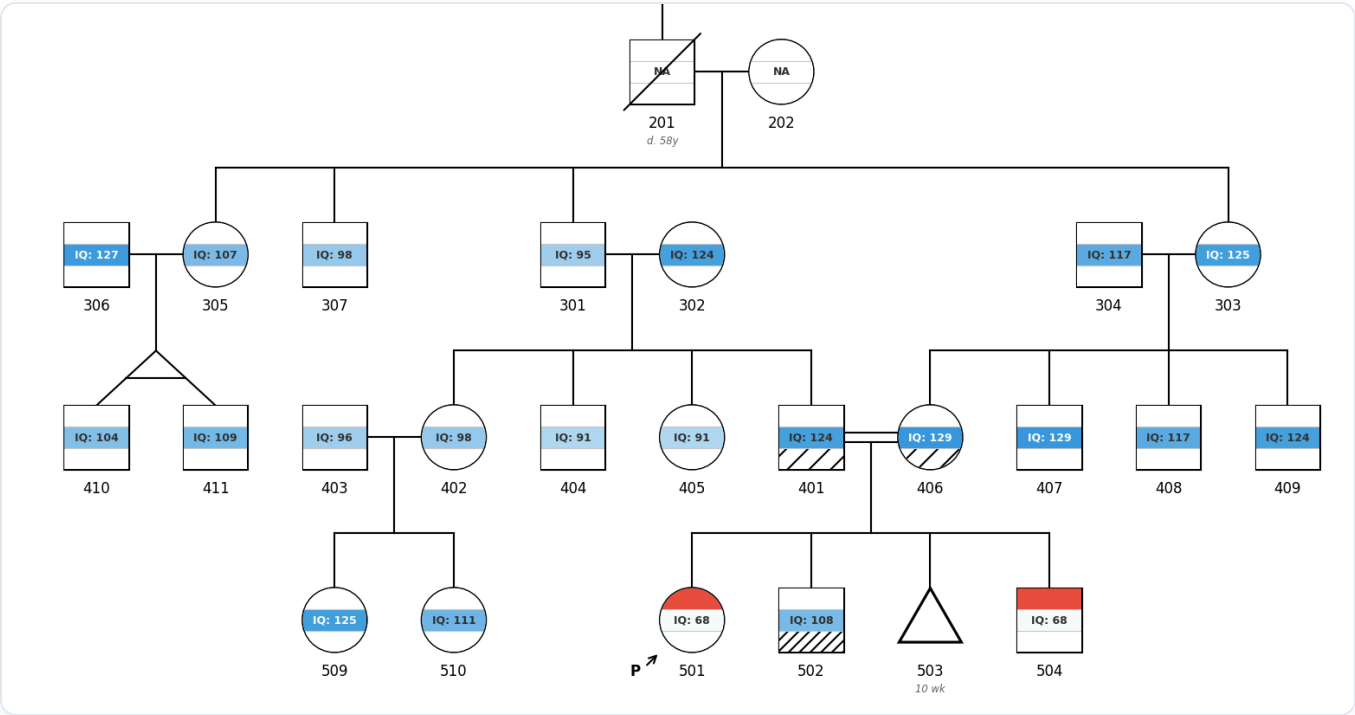
Annotations (gene names below IDs), diverging gradient (IQ), multi-PGS (colored markers), and zero-padded IDs

Five Generations with Loops (81 individuals)



`multigenerational.ped` *(synthetic): a first-cousin marriage (401 × 406) and a marriage between two branches (412 × 429), both drawn side by side with no duplicated individual, because each sibship is*

ordered so the branches of a loop meet. Binary, gradient and carrier bands, both kinds of twins, deceased individuals, a miscarriage and an adoption. Open it with `--interactive` to zoom in.



Detail: 401 and 406 are first cousins, grandchildren of 201. Their parents' sibships are ordered so they meet: 401 is the last of his sibship, 406 the first of hers. Their affected children (501, the proband, and 504) are below a double line; 502 is a carrier, 503 a miscarriage.

Error Handling

All pypedigree exceptions inherit from `PedigreeError`:

EXCEPTION	WHEN
`PedigreeParseError`	File parsing failures (bad format, missing columns, invalid values)
`PedigreeValidationError`	Data validation failures (duplicate IDs, circular references, sex mismatches)
`PedigreeLayoutError`	Layout computation failures

```
from pypedigree import parse_pedigree, PedigreeParseError

try:
    ped = parse_pedigree("invalid.ped")
except PedigreeParseError as e:
    print(f"Parse error: {e}")
```

Invalid Colors

Every color in a style is checked before drawing starts, and all offending values are reported together as a `PedigreeError` naming the option each came from:

```
PedigreeError: Invalid color(s) in style: outline_color = '#12345';
phenotype_colors['Epilepsy'] = '#zzzzzz'. Expected hex values like
'#e74c3c' or named colors like 'grey'.
```

Warnings

Non-fatal diagnostics use dedicated categories, so they can be filtered or escalated independently of other libraries' warnings.

WARNING	PARENT	WHEN EMITTED
`PedigreeWarning`	`UserWarning`	Base class for all pypedigree warnings

WARNING	PARENT	WHEN EMITTED
`PedigreeStyleWarning`	`PedigreeWarning`	A style definition contains something unusable, e.g. an unknown option
`PedigreeDataWarning`	`PedigreeWarning`	Pedigree data is questionable but still renderable, e.g. a father recorded as female

```
import warnings
from pypedigree import PedigreeDataWarning, parse_pedigree

# Treat questionable data as a hard failure
warnings.simplefilter("error", PedigreeDataWarning)
ped = parse_pedigree("family.ped")
```

An unknown key in a style file is ignored, but reported with a suggested spelling — silently dropping a typo is indistinguishable from the option having no effect:

```
PedigreeStyleWarning: Unknown style option 'phenotype_color' ignored;
did you mean 'phenotype_colors'?
```

The CLI captures all of these and prints them as `Warning: <message>` on stderr.